

Bachelorarbeit

Ansatz zur Optimierung einer CI-Pipeline mit reproduzierbaren lokalen Builds

Ruben Leander Rosenmeyer

Gutachter: Prof. Dr. Peter Thiemann

Betreuer: Dr. Michael Sperber

Albert-Ludwigs-Universität Freiburg

Technische Fakultät

Institut für Informatik

13. Juli 2026

Bearbeitungszeit

13. 04. 2026 – 13. 07. 2026

Gutachter

Prof. Dr. Peter Thiemann

Betreuer

Dr. Michael Sperber

ERKLÄRUNG

Hiermit erkläre ich, dass ich diese Abschlussarbeit selbständig verfasst habe, keine anderen als die angegebenen Quellen/Hilfsmittel verwendet habe und alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, als solche kenntlich gemacht habe.

Darüber hinaus erkläre ich, dass diese Abschlussarbeit nicht, auch nicht auszugsweise, bereits für eine andere Prüfung angefertigt wurde.

Ort, Datum

Unterschrift

Zusammenfassung

Diese Bachelorarbeit beschäftigt sich mit der Erstellung eines Tools, welches anhand der Ausgabe eines vorherigen Testlaufes entscheiden kann, welche Testcases innerhalb eines Projektes bereits abgelaufen sind, und welche nicht. Dabei soll das Tool unabhängig vom dem oder den verwendeten Testframework(s) sein, und sich in eine bestehende CI-Pipeline integrieren lassen, auch hier unabhängig von der verwendeten Software.

Dadurch soll die Gesamtzahl der Testfälle, die abgelaufen werden müssen, reduziert werden.

Hierfür werden die Anforderungen an ein Projekt, in das das Tool integriert werden kann, beschrieben und erklärt, sowie unterschiedliche Testausgabeformate hinsichtlich ihrer Eignung miteinander verglichen und diskutiert.

Inhaltsverzeichnis

Zusammenfassung	ii
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
2 Hintergrund	3
2.1 Testausgabeformate	3
2.2 JUnit XML im Detail	3
3 Approach	4
3.1 Problem Definition	4
3.2 First Part of the Approach	4
3.3 N-th Part of the Approach	4
4 Experiments	5
5 Related Work	6
6 Conclusion	7
6.1 Möglichkeiten zur Verbesserung	7
Bibliography	7

1 Einleitung

1.1 Problemstellung

Jedes größere Softwareprojekt benötigt eine Vielzahl von Tests, um die Qualität der Software zu sichern. Dabei werden die einzelnen Tests für gewöhnlich in verschiedene Klassen unterteilt, die verschiedene Abschnitte der Software abdecken. Die meisten gängigen Testframeworks bieten die Möglichkeit, einzelne Testklassen und/oder Testfälle gezielt auszuführen, ohne die restlichen Testfälle dabei mit auszuführen.

Doch diese Praxis birgt zum einen die Gefahr, dass manche Tests vom Entwickler vergessen werden können. Andererseits ist es manchmal auch gewünscht, dass nur gewisse Tests ausgeführt werden, beispielsweise bei einer langen Laufzeit der Tests. Dies kann dazu führen, dass die Tests nicht in ihrer Gesamtheit ausgeführt werden, und Fehler unentdeckt im Code verbleiben können.

1.2 Zielsetzung

Das Ziel des im Rahmen dieser Bachelorarbeit entwickelten Tools ist, eine verlässliche Aussage darüber treffen zu können, welche Tests in der aktuellen Iteration bereits ausgeführt wurden, und genau anzugeben, welche Testfälle zum Erreichen einer 100%igen Testabdeckung noch ausgeführt werden müssen. Dabei soll die Wahl des Testframeworks möglichst keine Rolle spielen.

Außerdem soll das Tool in eine bestehende CI-Pipeline integriert werden können.

2 Hintergrund

2.1 Testausgabeformate

2.2 JUnit XML im Detail

3 Approach

3.1 Problem Definition

3.2 First Part of the Approach

3.3 N-th Part of the Approach

4 Experiments

5 Related Work

6 Conclusion

6.1 Möglichkeiten zur Verbesserung

In seiner aktuellen Form kann das Tool nicht entscheiden, ob die XML-Ausgabe der Tests auch tatsächlich aus dem aktuellen Stand des Codes entstammt, oder ob im Nachhinein Änderungen am Code vorgenommen wurden, was die Aussagekraft des Tools stark beeinträchtigt.

Ein anderes Problem ist, dass das Tool alle Tests, die nicht in der letzten Iteration abgelaufen sind, als nicht abgelaufen betrachtet, auch wenn einige Testfälle in einer vorherigen Iteration ausgeführt sein könnten, und die Änderungen, die seither im Code gemacht wurden diese Tests überhaupt nicht betreffen.

Eine Möglichkeit, dieses Problem zu beheben, wäre die Integration eines neuen Tools, welches eine Selective Regression Testing implementiert, und so dem Tool rückmelden kann, welche Tests tatsächlich neu ausgeführt werden müssen, und bei welchen der vorherige Stand ausreicht.

Literaturverzeichnis

